

Cell Decomposition and Coverage Planning

Robots in Context

Leon Bodenhagen lebo@mmmi.sdu.dk

Associate Professor

SDU Robotics, Maersk Mc-Kinney Moller Institute



Agenda

- Presentation for PE2 (Bug)
 - Jakob
- Presentations for PE3 (Line Fitting)
 - Dominik
 - Povl
- Presentations for PE4 (GVD + Brushfire)
 -
- Lecture 5 - Search + Coverage



Agenda

- Presentation for PE2 (Bug)
 - Jakob
- Presentations for PE3 (Line Fitting)
 - Dominik
 - Povl
- Presentations for PE4 (GVD + Brushfire)
 -
- Lecture 5 - Search + Coverage



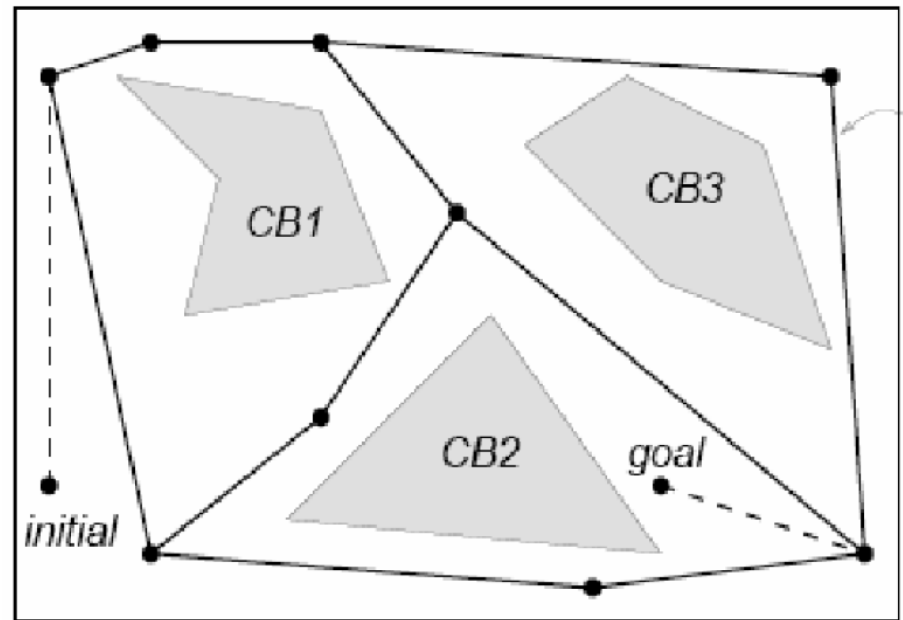
Content

- Roadmaps: A* search
- ☕
- Cell Decomposition
- Trapezoidal Decomposition
- Coverage Planning
 - Based on Trapezoidal Decomposition
 - Boustrophedon Decomposition
 - Morse Decomposition
 - Brushfire decomposition
 - Sensor based



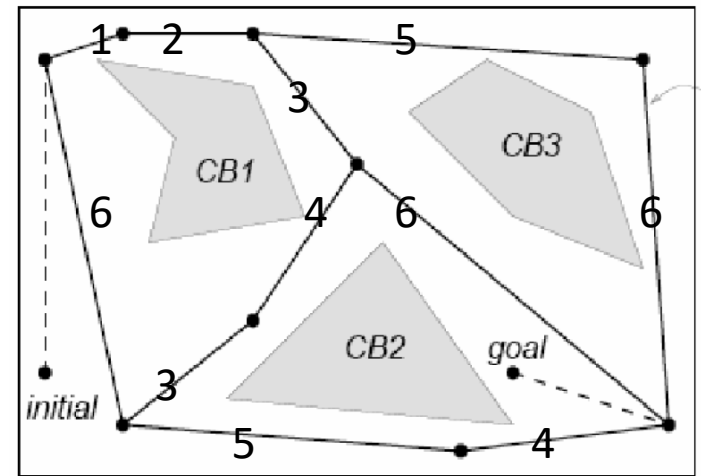
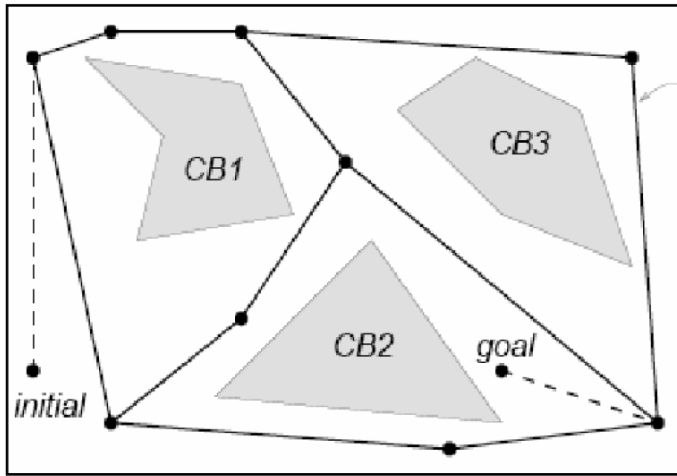
Roadmaps

Basic idea: Capture the connectivity of the free space by a graph.



Searching for a Path through a Roadmap

- Unweighted vs. weighted graph

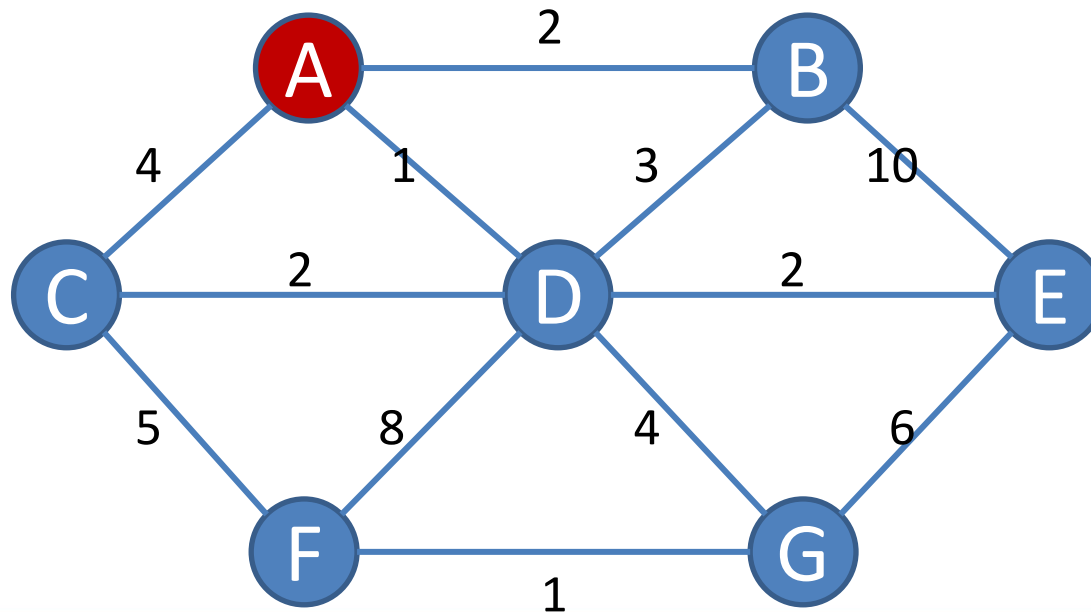


- Shortest path search in a graph
 - Dijkstra's Algorithm
 - A*
 -



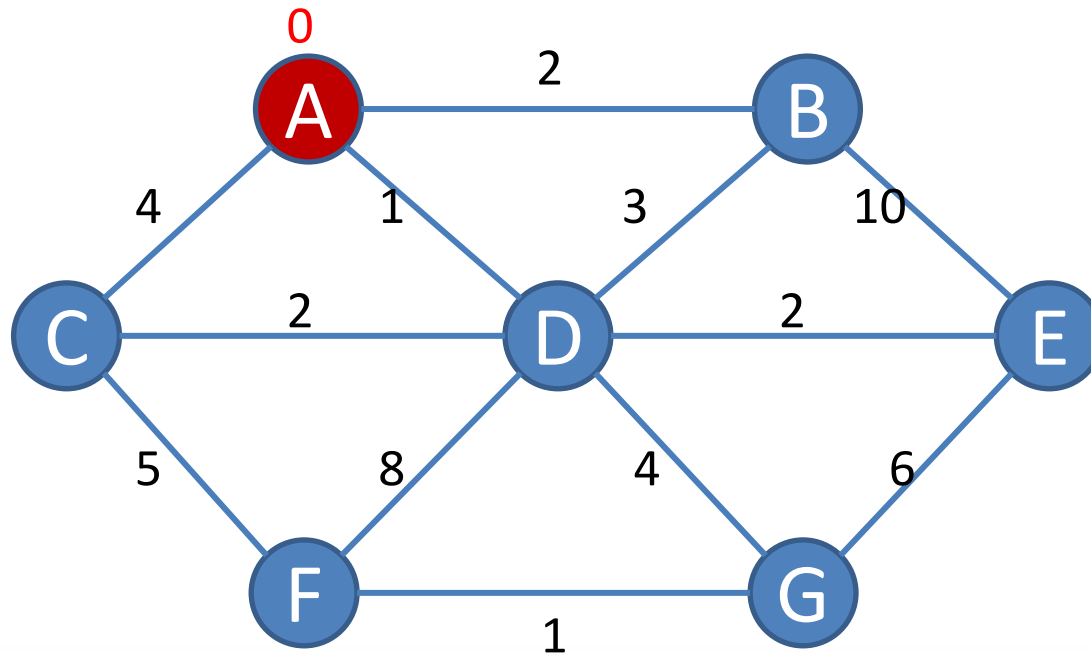
Weighted Shortest Path

- Dijkstra's Algorithm
 - Start at node A



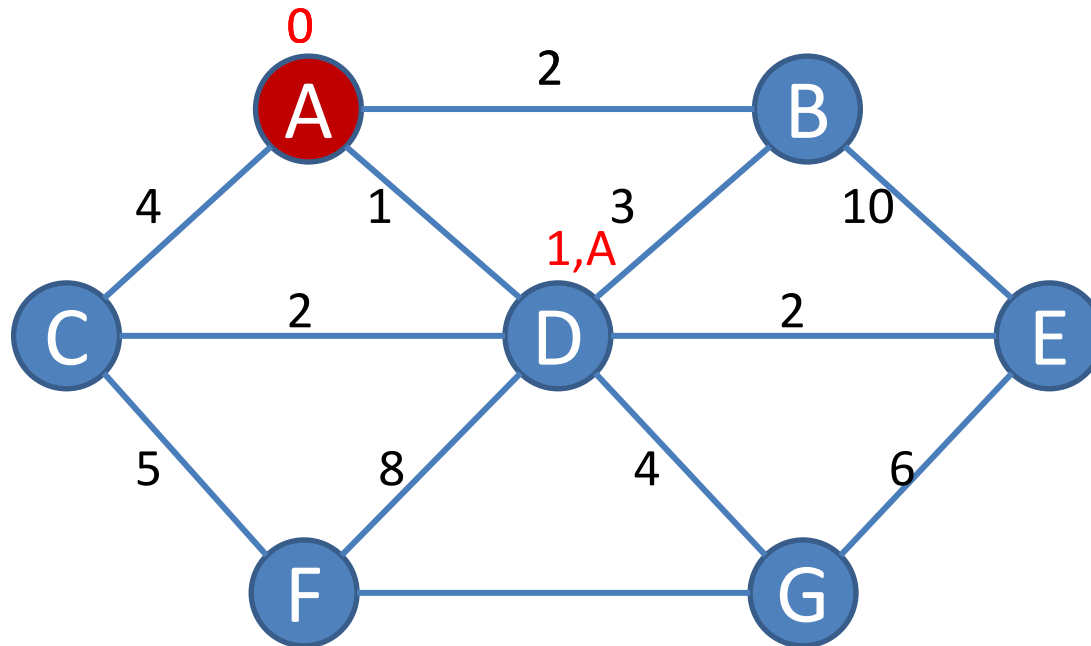
Weighted Shortest Path

- Dijkstra's Algorithm
 - Mark A with 0 and go to the “cheapest” node



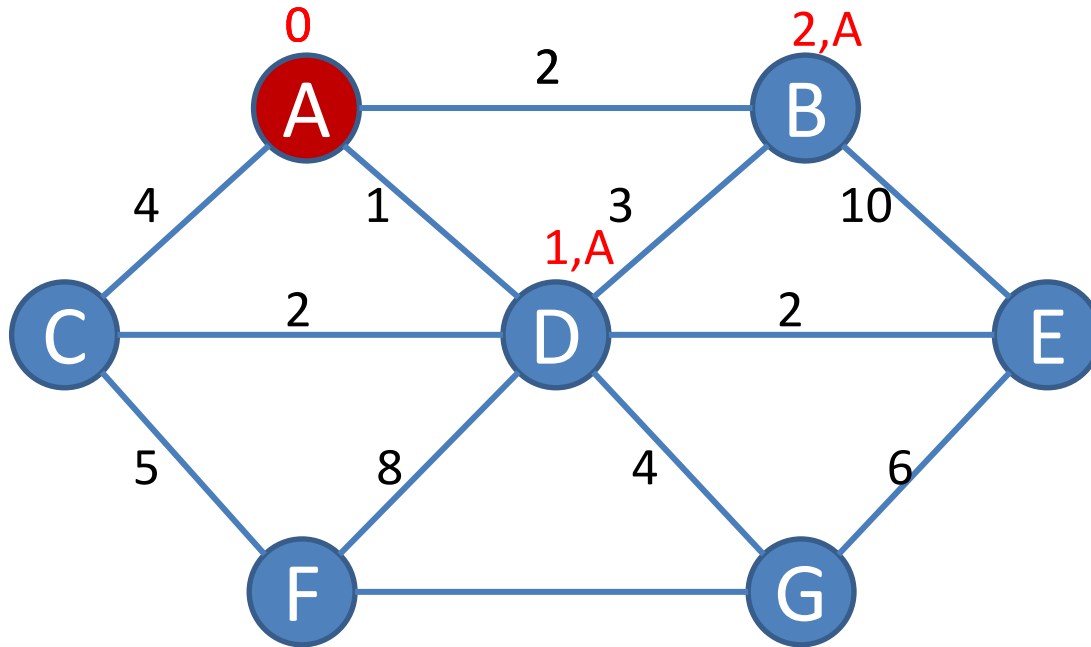
Weighted Shortest Path

- Dijkstra's Algorithm
 - Mark node with cost
 - Go to the new “Cheapest” node



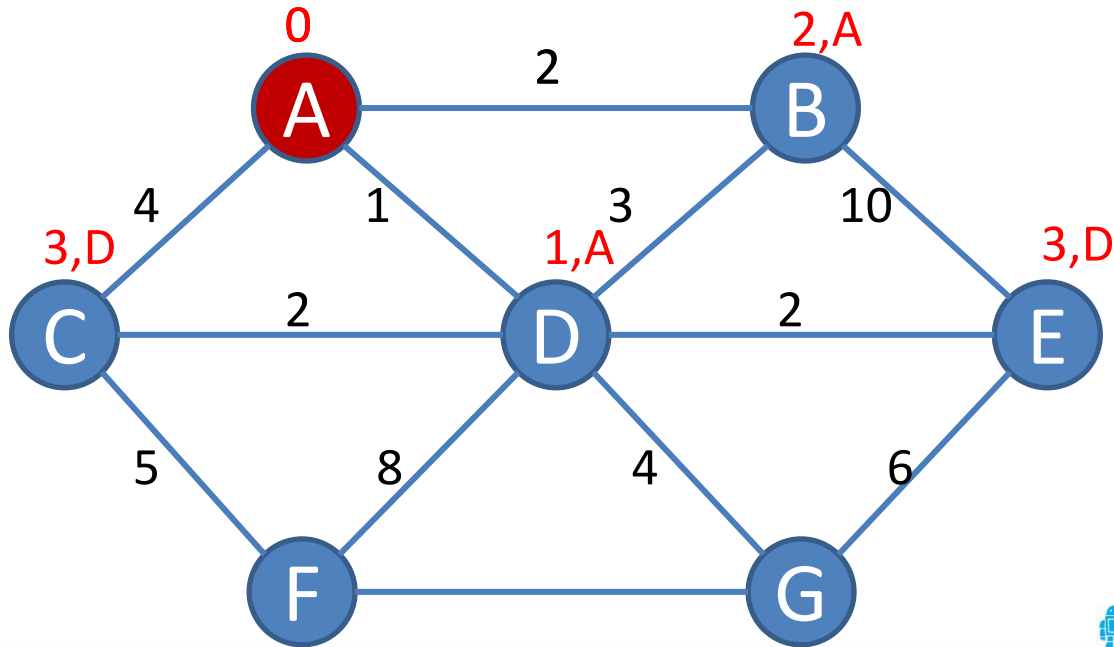
Weighted Shortest Path

- Dijkstra's Algorithm
 - Mark node with cost
 - Go to the new "Cheapest" node



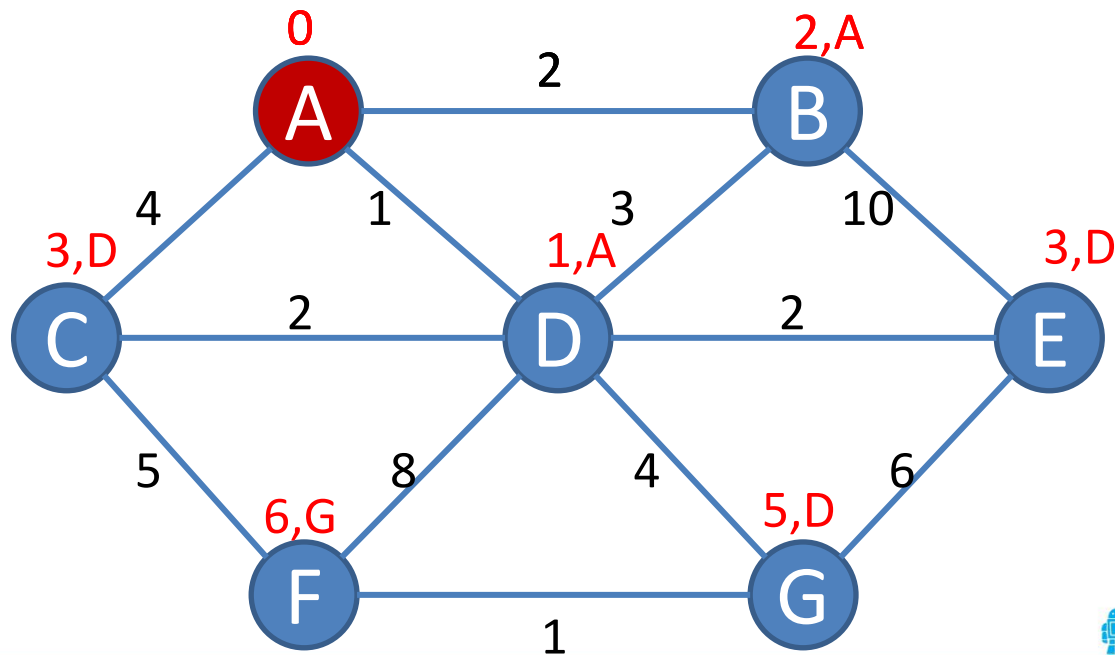
Weighted Shortest Path

- Dijkstra's Algorithm
 - Mark node with cost
 - Go to the new "Cheapest" node



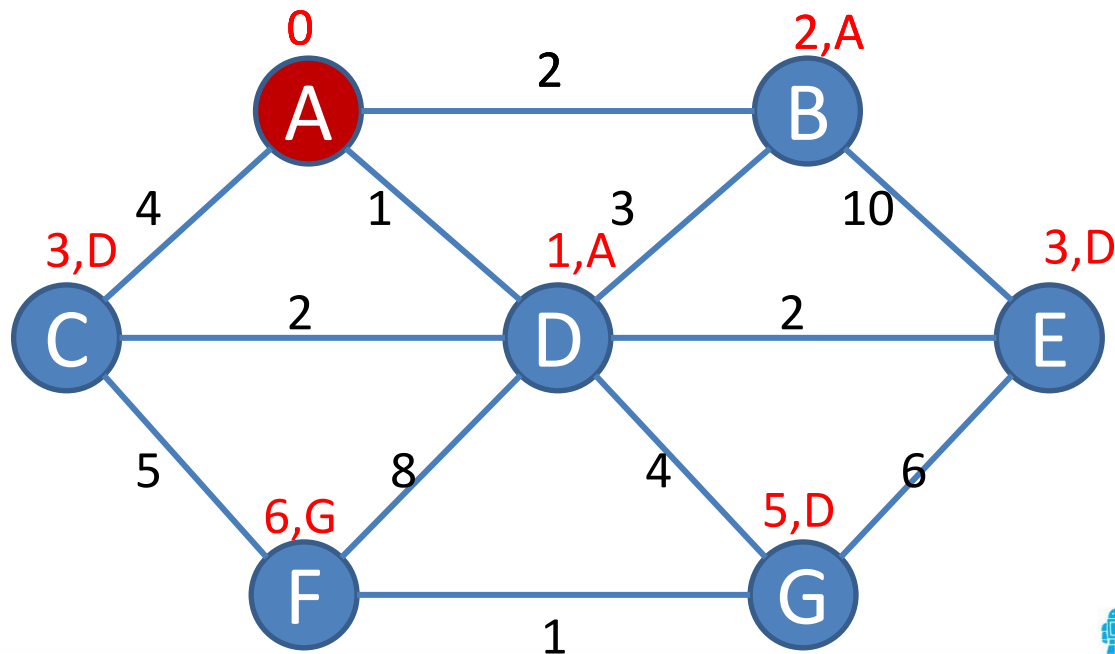
Weighted Shortest Path

- Dijkstra's Algorithm
 - Continue like this until all points are marked.



Weighted Shortest Path

- Dijkstra's Algorithm
 - To find a path back to A from a given node use the markings to see which node to go to.



- Why do we search all configurations? Could we do this more clever?



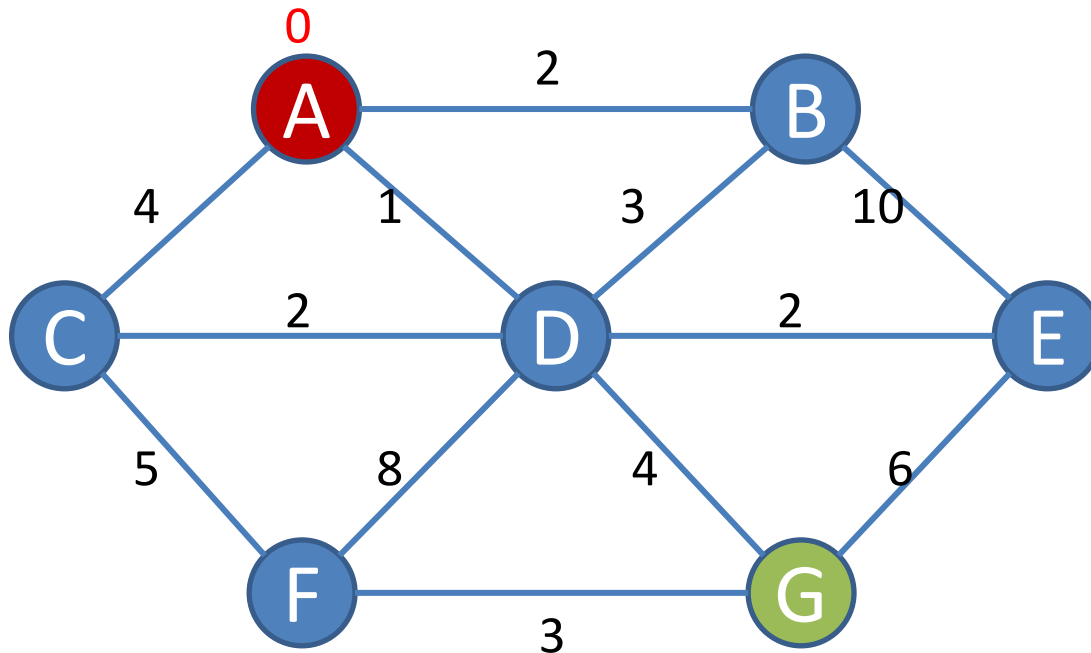
A*

- Use a **heuristic** to guide the search.
- We need to know both the start and goal.
 - In each step consider not only the cheapest solution, but also the expected remaining distance to the goal minimizing $f(n) = g(n) + h(n)$.
 - The **heuristic**, h , needs to be **optimistic** such that it does not estimate a longer distance to the goal than the actual distance.
 - A heuristic could be the distance $d(q, q_{goal})$



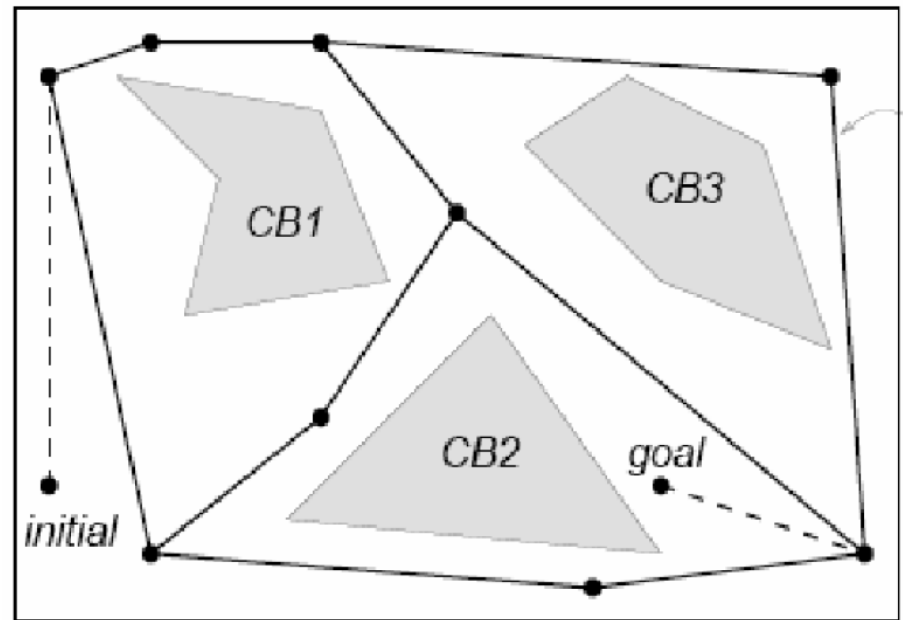
A*

- Explore nodes with the lowest estimated cost $f(n) = g(n) + h(n)$, with the actual cost accumulated so far, g , and the estimated remaining distance, h



Roadmaps

Basic idea: Capture the connectivity of the free space by a graph.



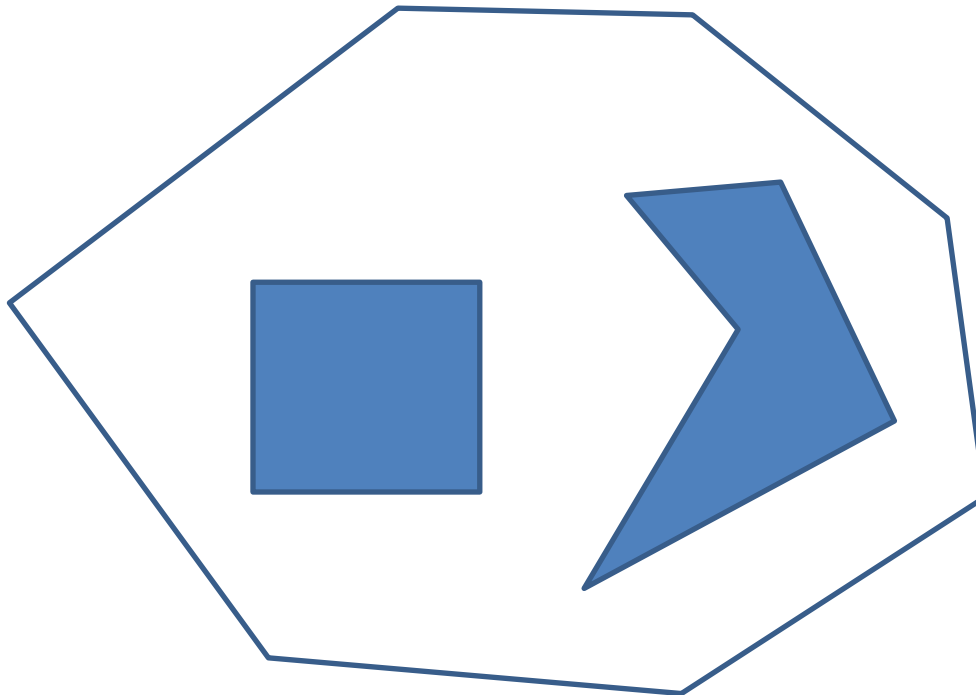
Cell Decomposition

- Similarities with roadmaps
 - Can be used for path planning
- Different objective:
 - Coverage



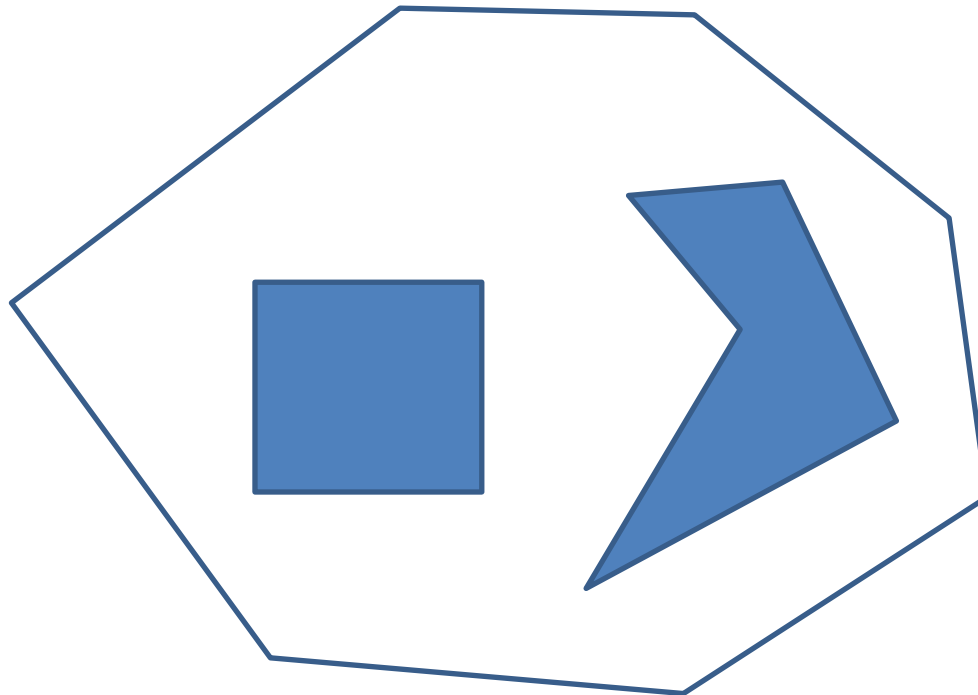
Cell Decomposition

- Given a workspace with obstacles we can decompose it into a number of convex cells.



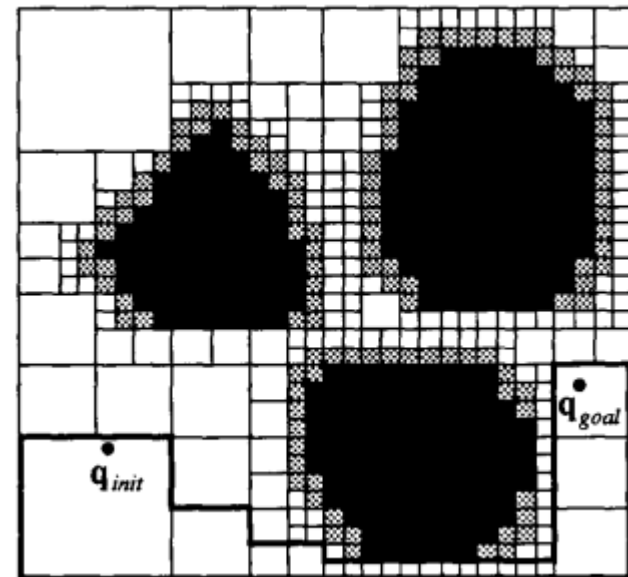
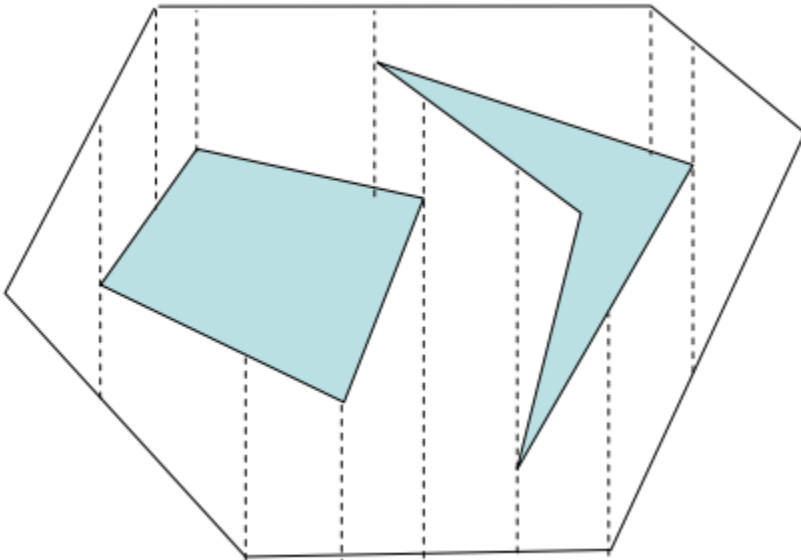


- Given a workspace with obstacles we can decompose it into a number of **?convex?** cells.



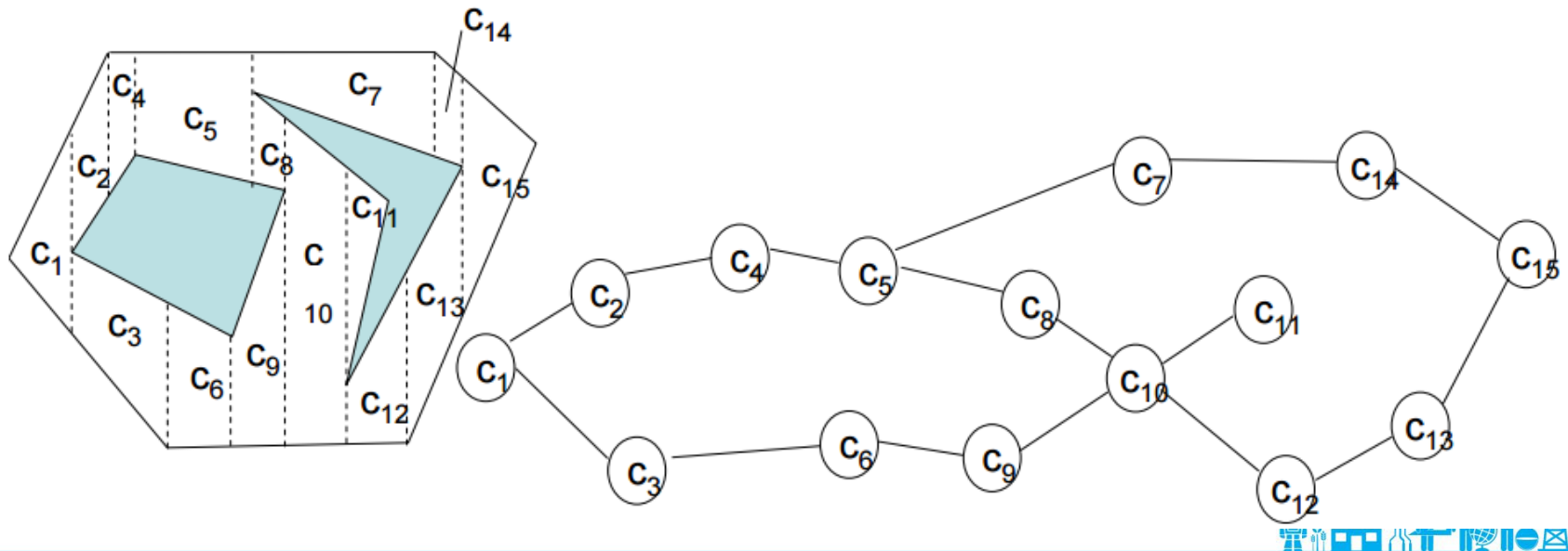
Cell Decomposition

- Exact Decomposition
- Approximate Decomposition



Adjacency Graph

- Each cell has a node
- Edges between nodes if the two cells share a common boundary



Path Planning using Adjacency Graphs

- Determine in which cell the start and goal are located.
- Search through the adjacency graph to find a path.
- Due to the convex cells you can always go on a straight line from a point in the cell to a point on the boundary.

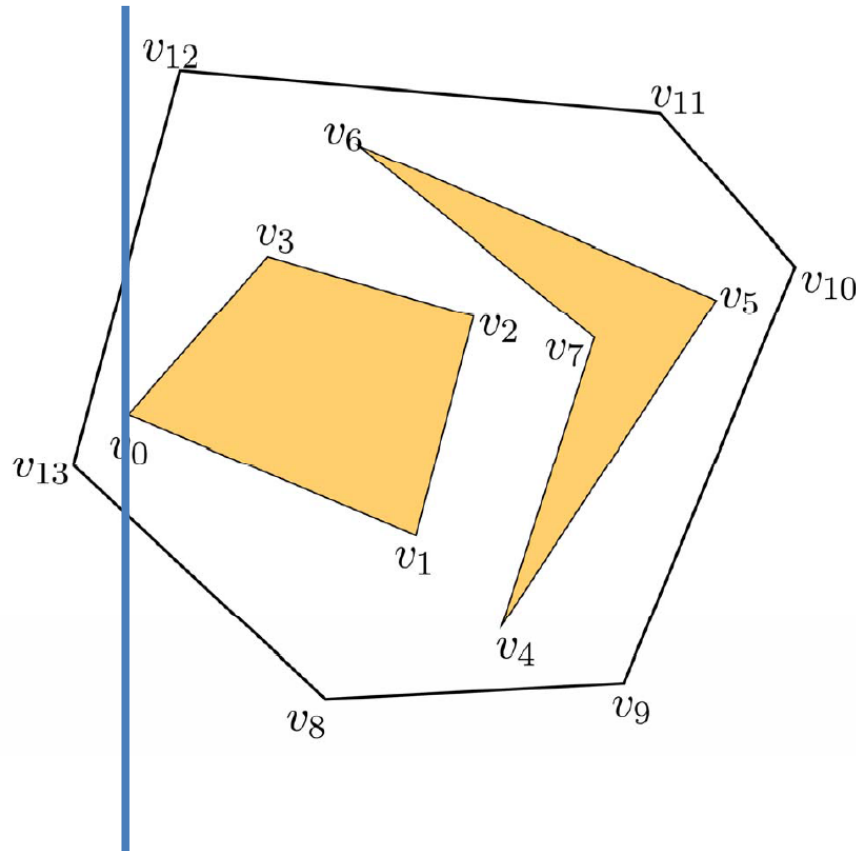


- Sweep Line



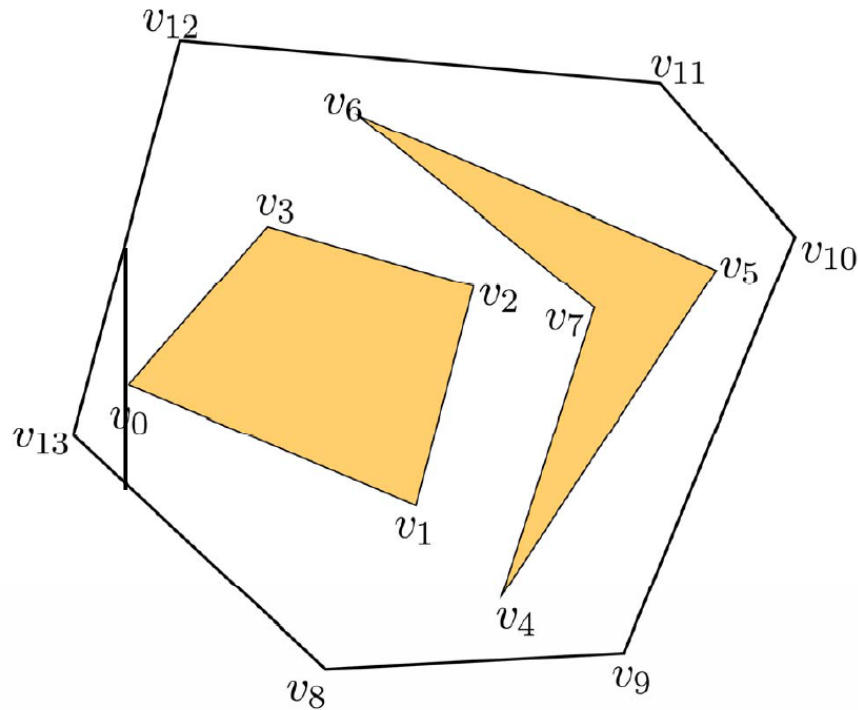
Trapezoidal Decomposition

- Stop when meeting a vertex



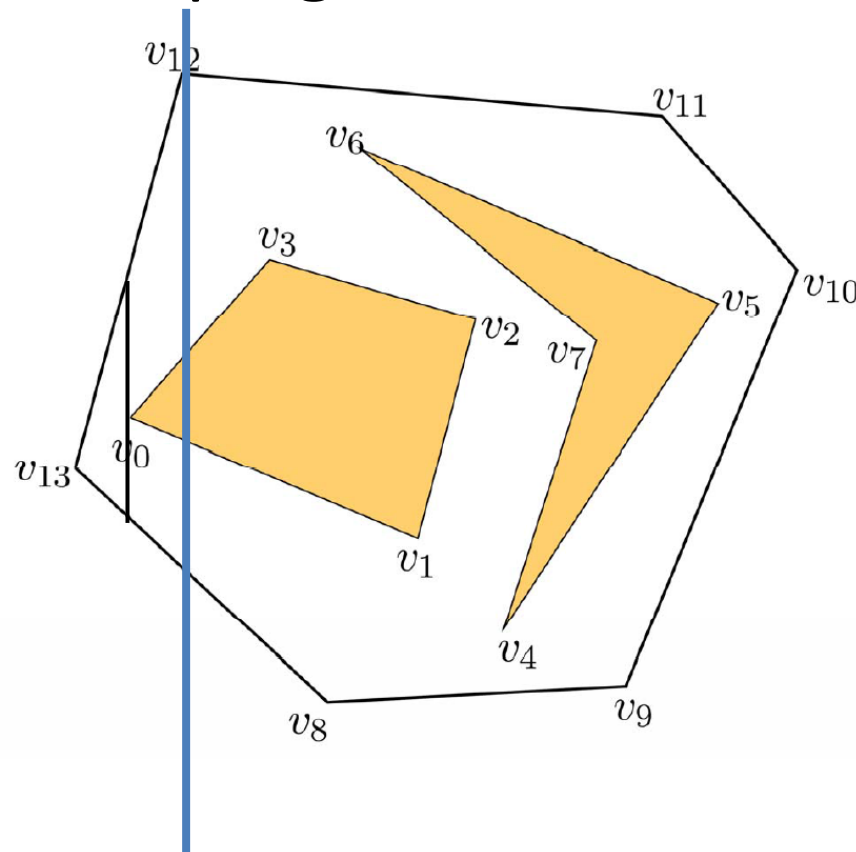
Trapezoidal Decomposition

- Create the cell with a vertical line



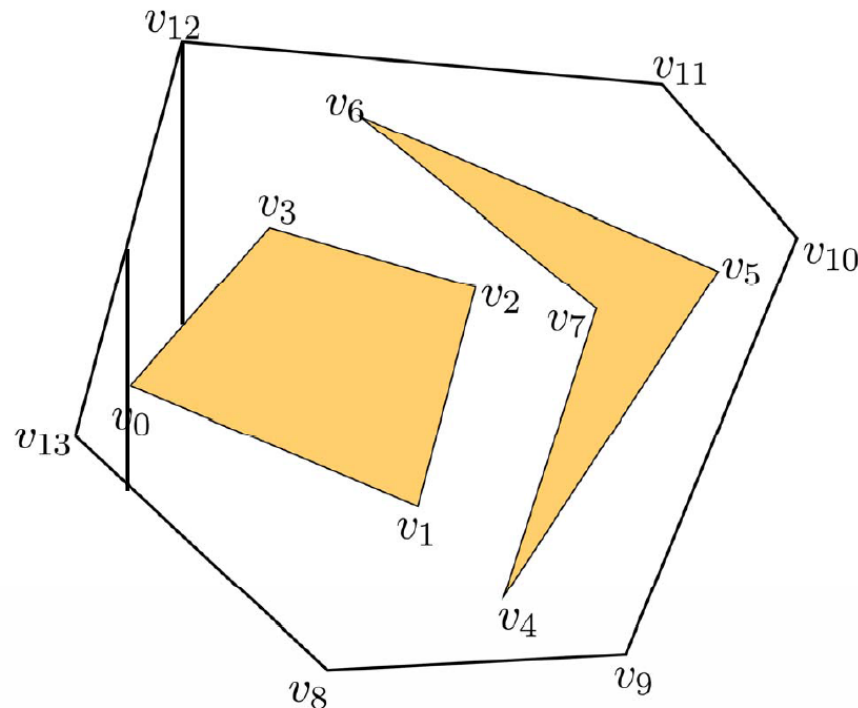
Trapezoidal Decomposition

- Continue sweeping line until next vertex



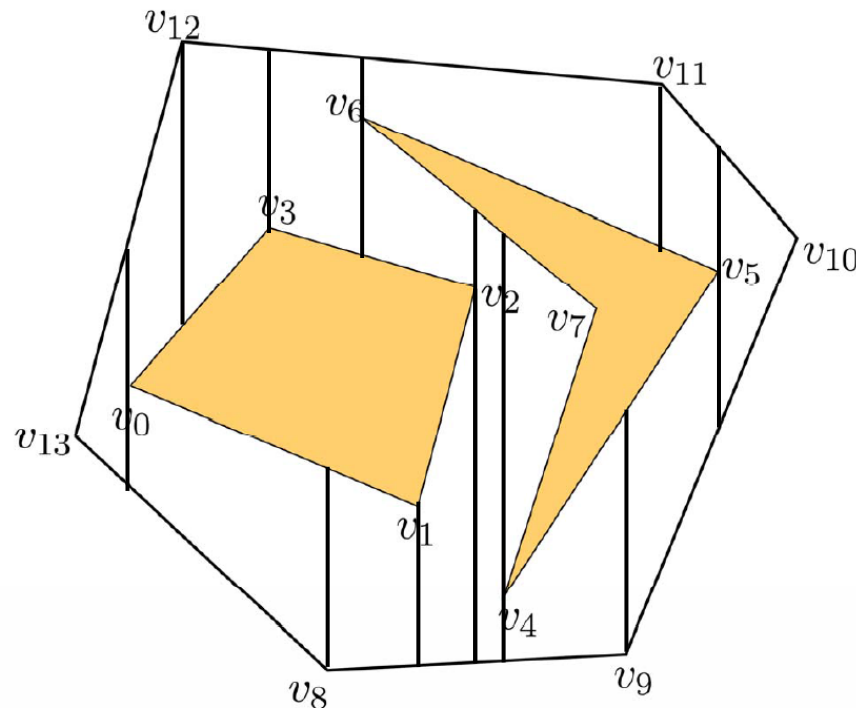
Trapezoidal Decomposition

- Create cell moving from the vertex to obstacle



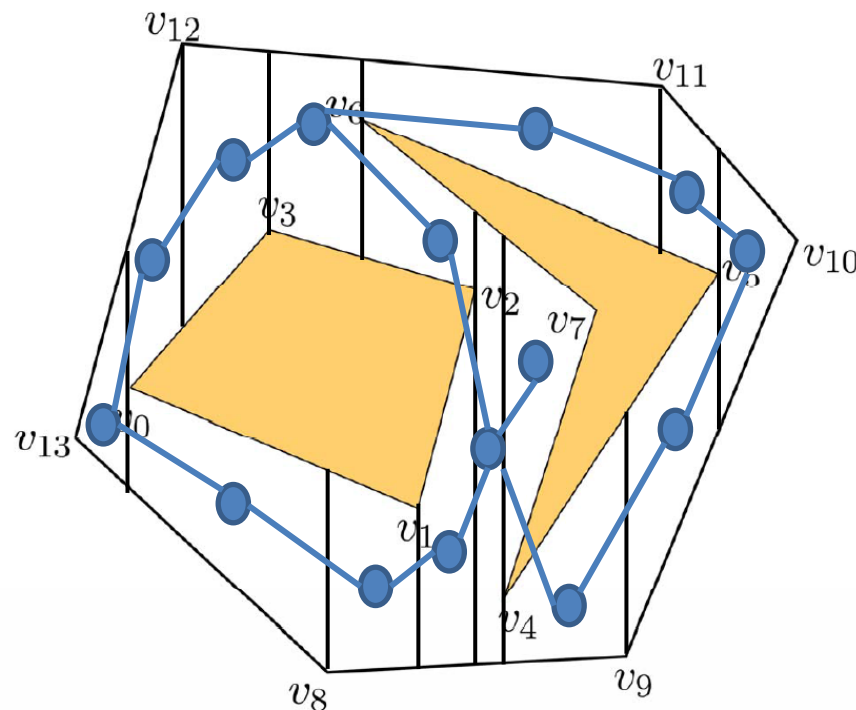
Trapezoidal Decomposition

- Continue sweeping and adding



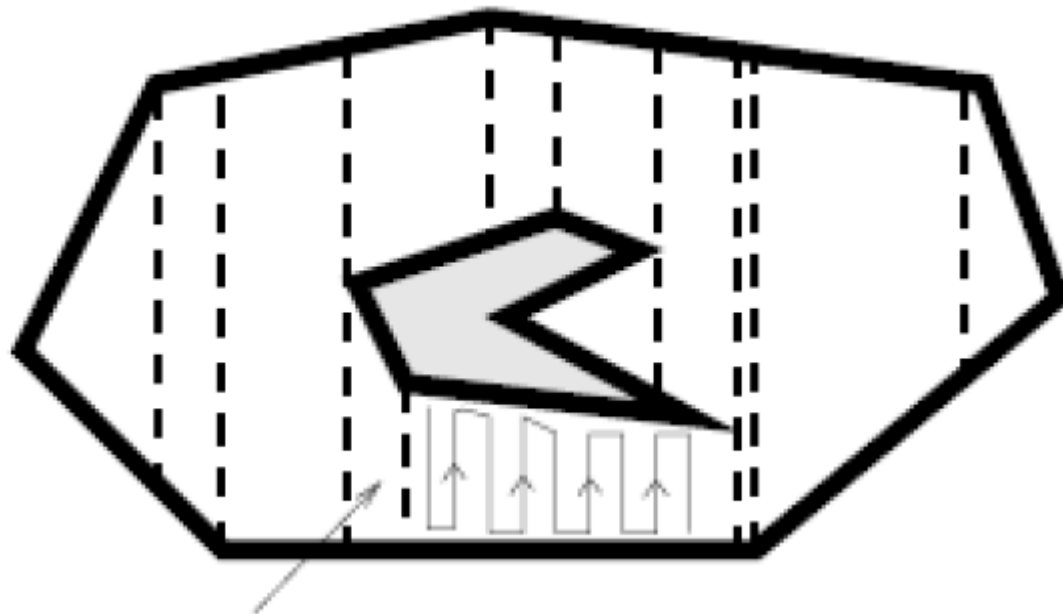
Trapezoidal Decomposition

- Creating nodes and edges



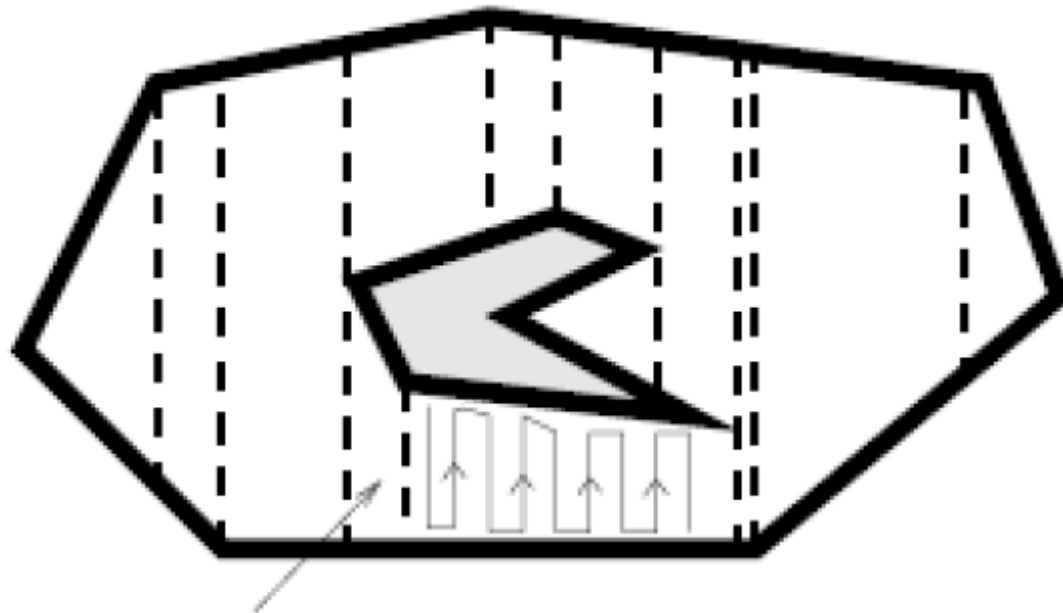
Coverage Planning

- Having decomposed into cells we can run through all cells and apply a simple pattern for covering.



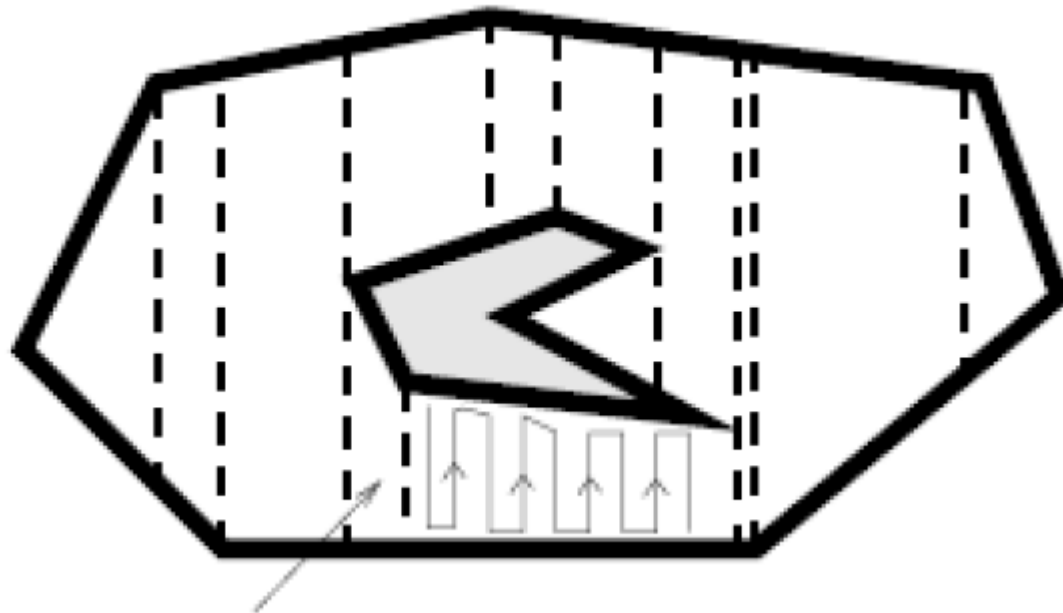
Coverage Planning

- Having decomposed into cells we can run through all cells and apply a simple pattern for covering.
- Problem: Can be very suboptimal!



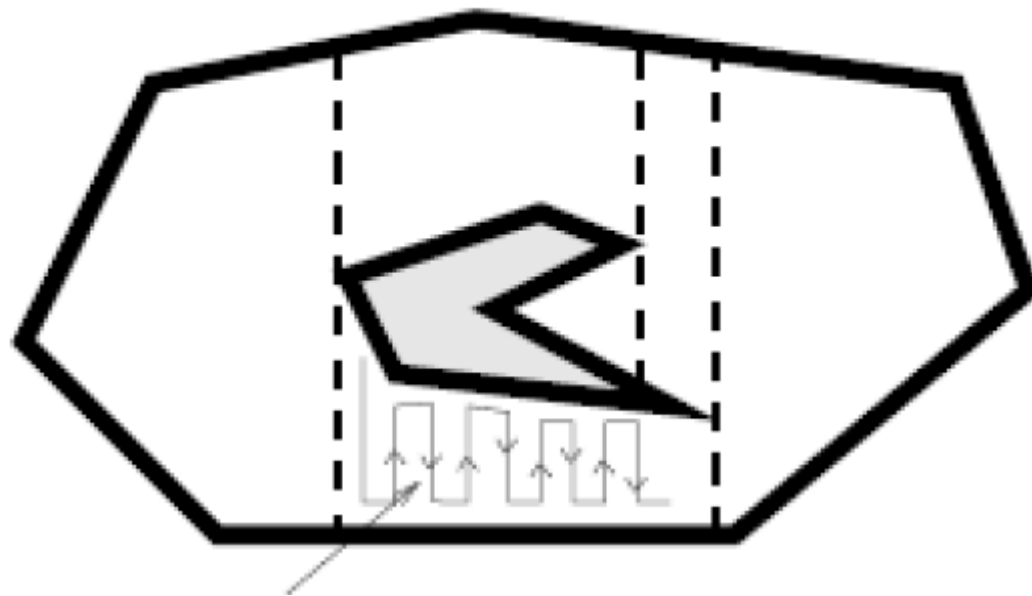
Coverage Planning

- Having decomposed into cells we can run through all cells and apply a simple pattern for covering.
- Problem: Can be very suboptimal! What is optimal?



Coverage Planning

- We can achieve better performance by grouping cells together.

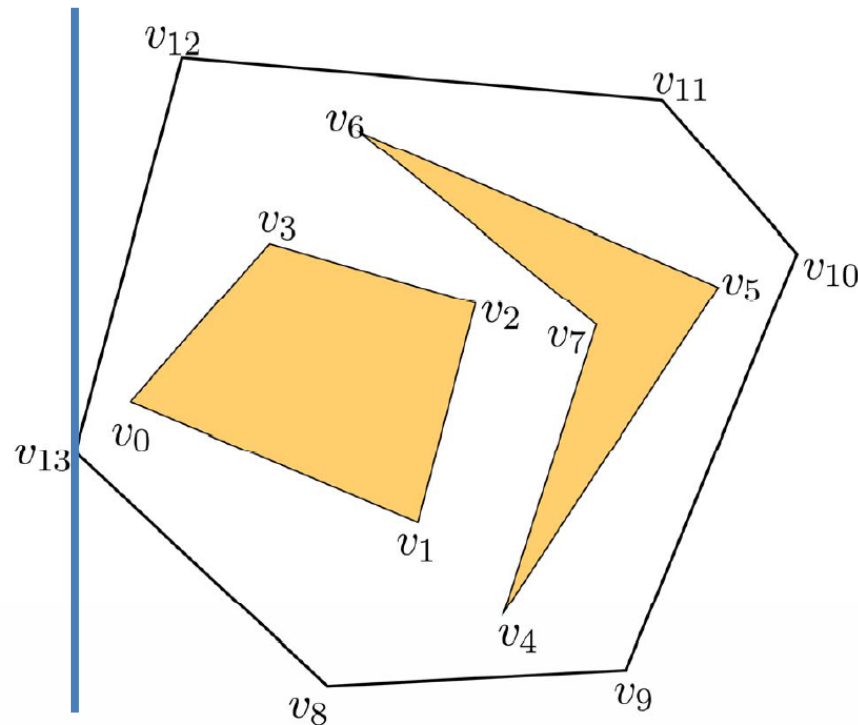


Boustrophedon Decomposition

- Follows the principles from Trapezoidal Decomposition, but only separates at critical points where the sweep line can be extended in both directions.

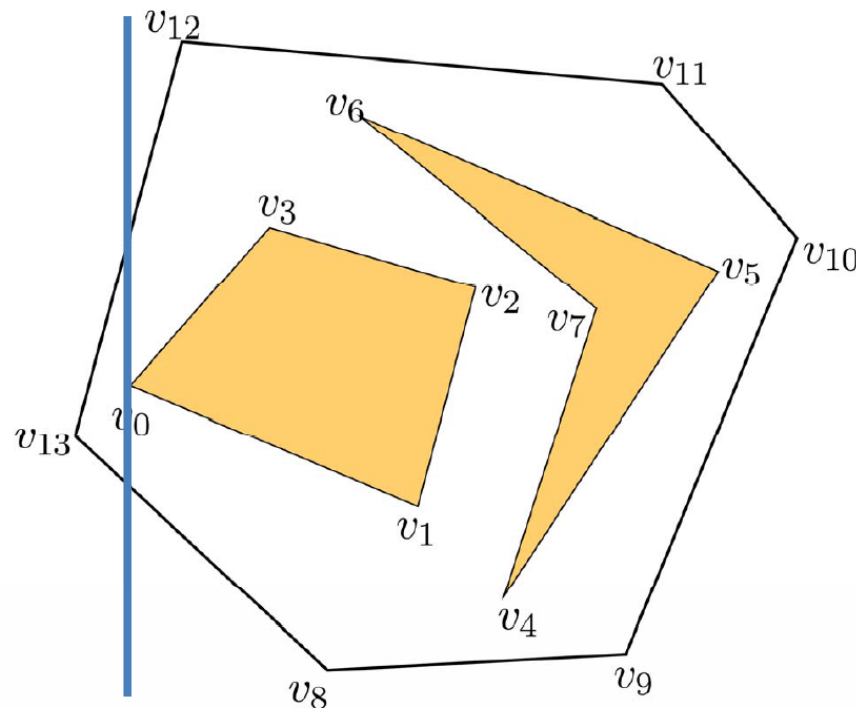


- Sweep Line



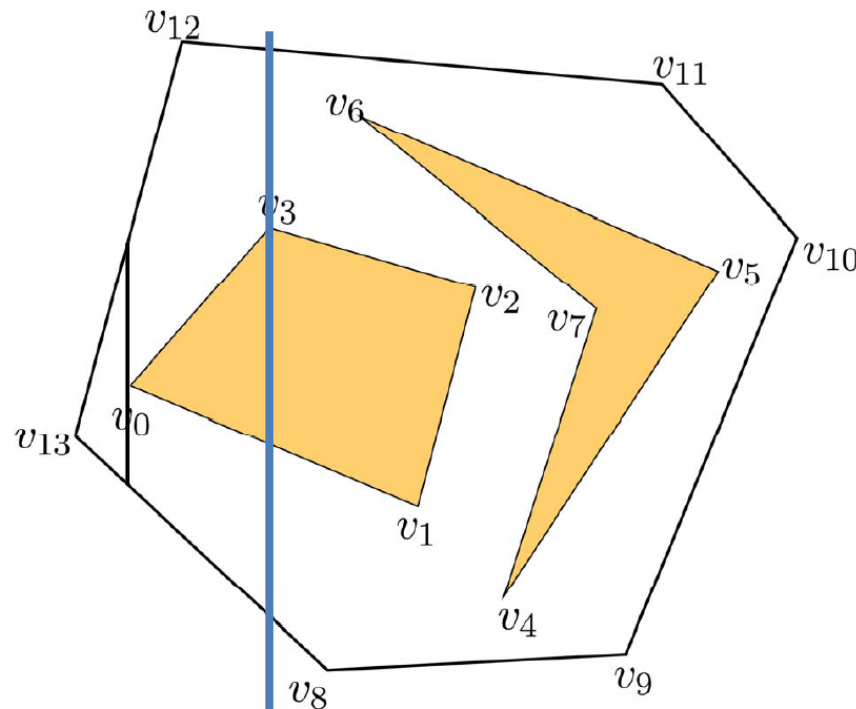
Boustrophedon Decomposition

- Stop when meeting a vertex. Check if it can be extended in both directions



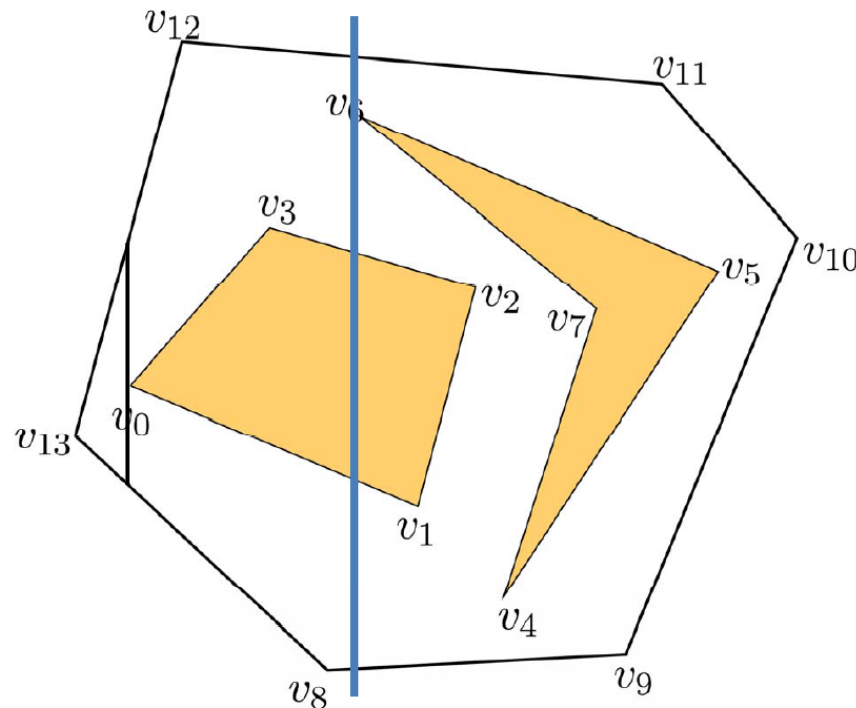
Boustrophedon Decomposition

- Continue sweeping line until next vertex



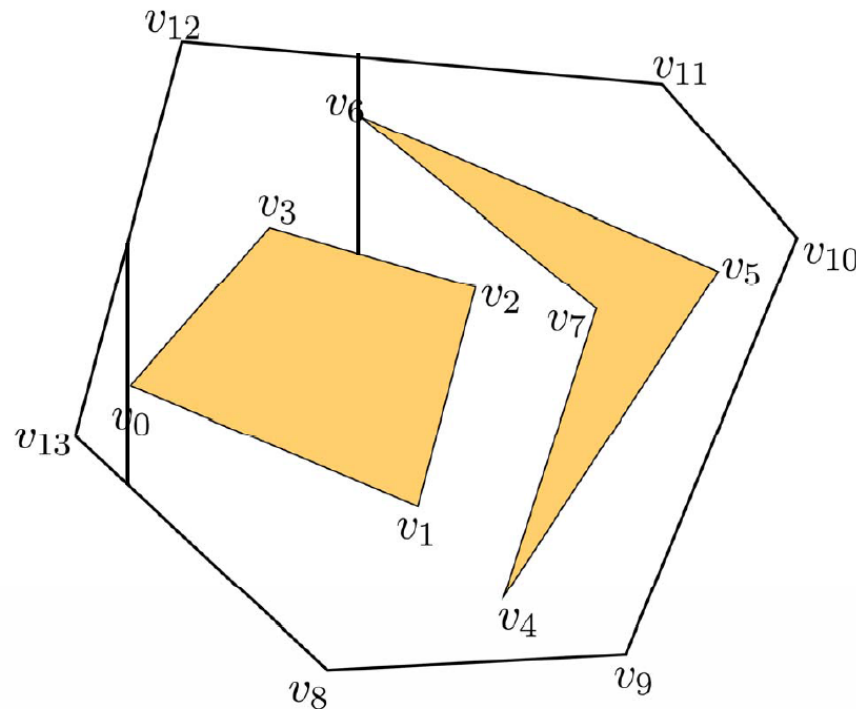
Boustrophedon Decomposition

- Continue sweeping line until next vertex

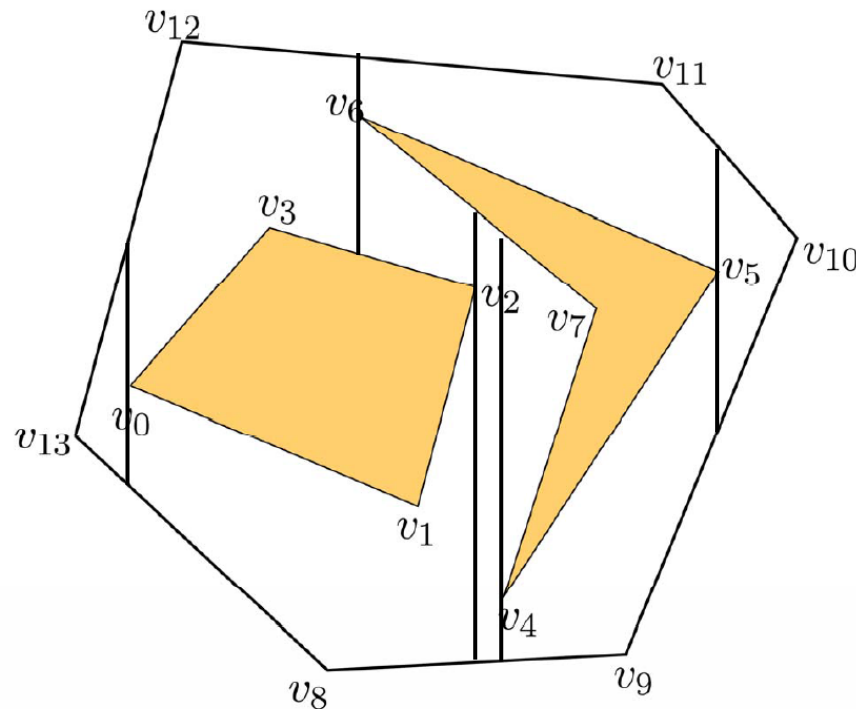


Boustrophedon Decomposition

- Stop when meeting a vertex. Check if it can be extended in both directions

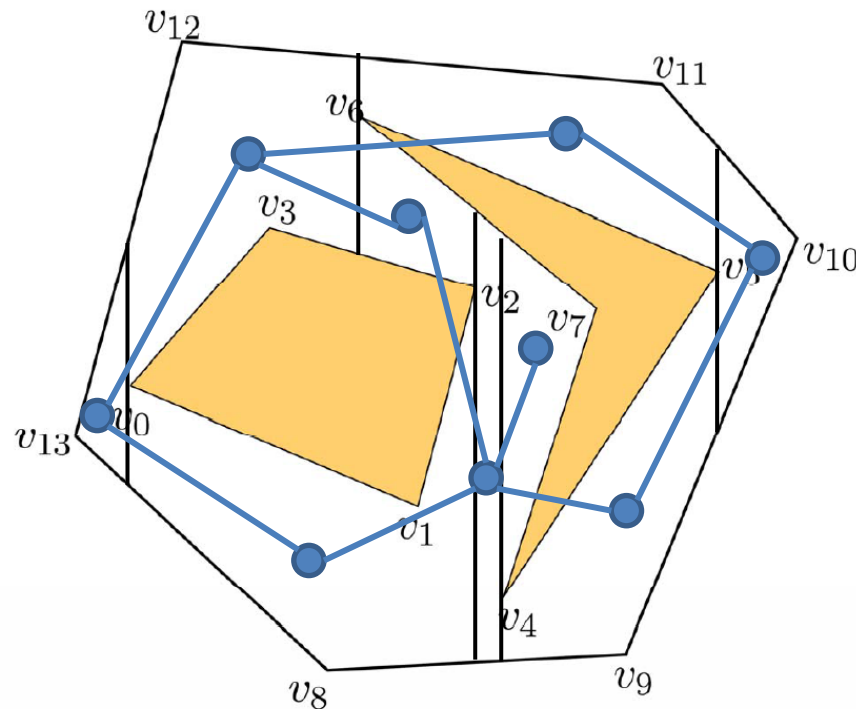


- Continue until all vertices are visited



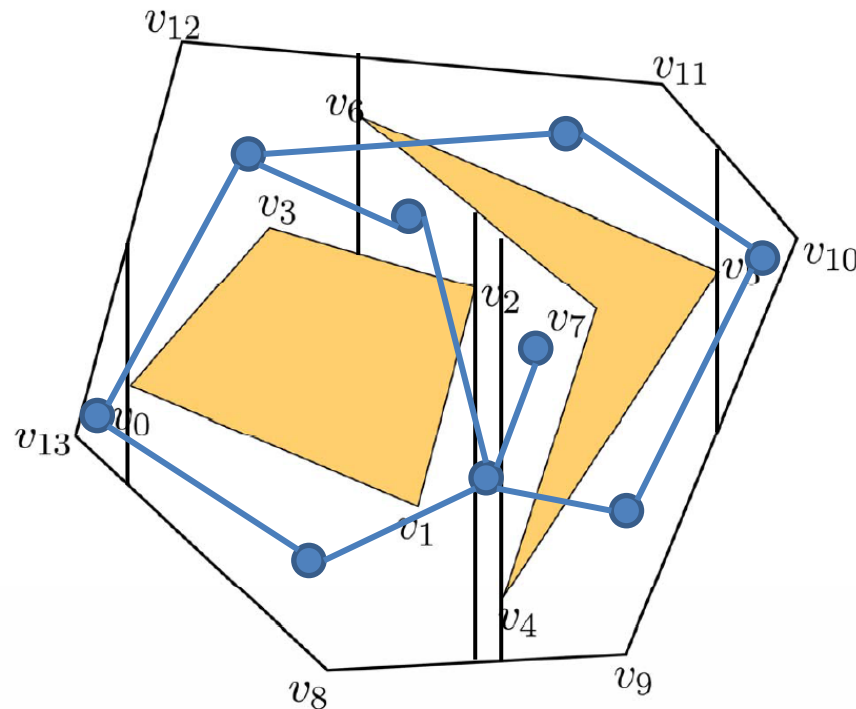
Boustrophedon Decomposition

- Continue until all vertices are visited



Boustrophedon Decomposition

- Do a depth first search to visit all nodes.



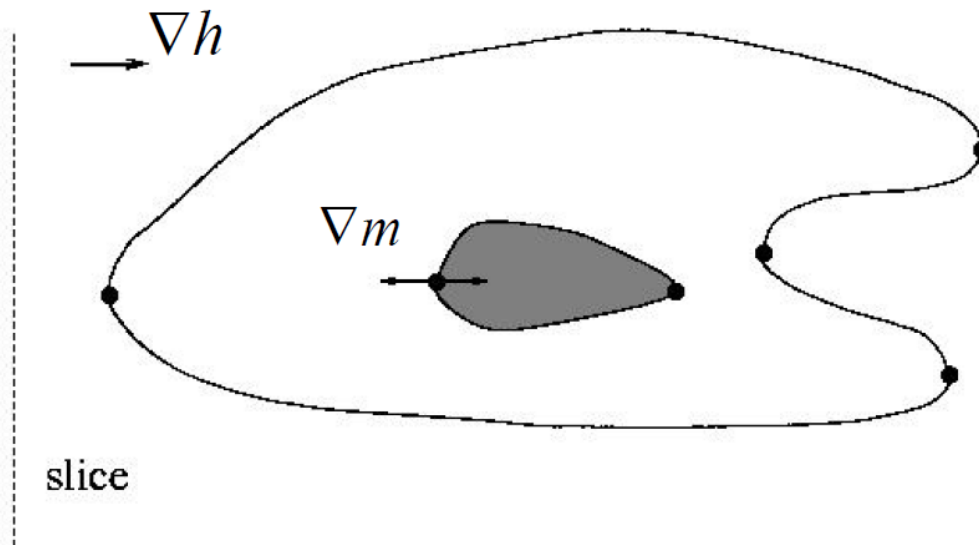
Morse Decomposition

- Trapezoidal Decomposition is only defined for polygons.
- Based on Morse functions described in Chapter 5 and related to the Canny Roadmap Algorithm



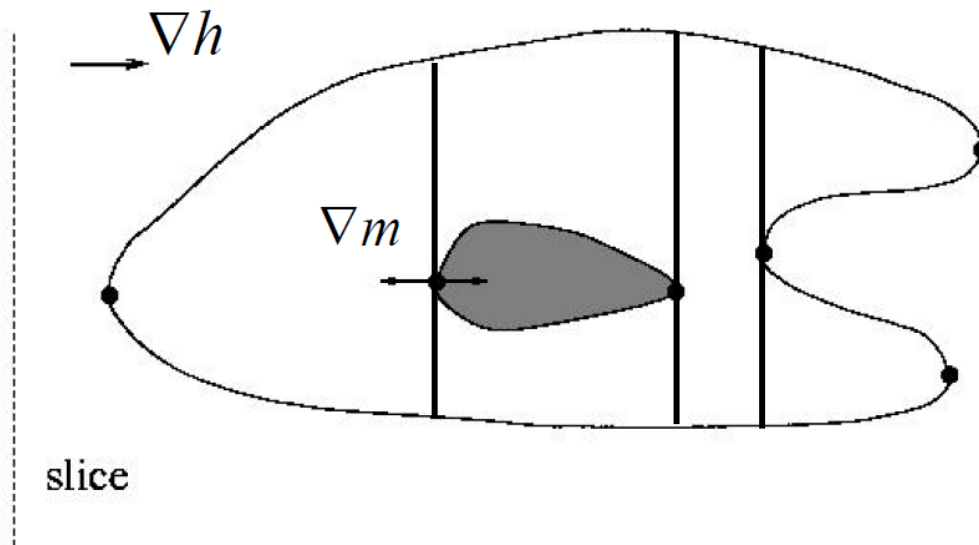
Morse Decomposition

- At a critical point the gradient of the slice function and the obstacle are equal.



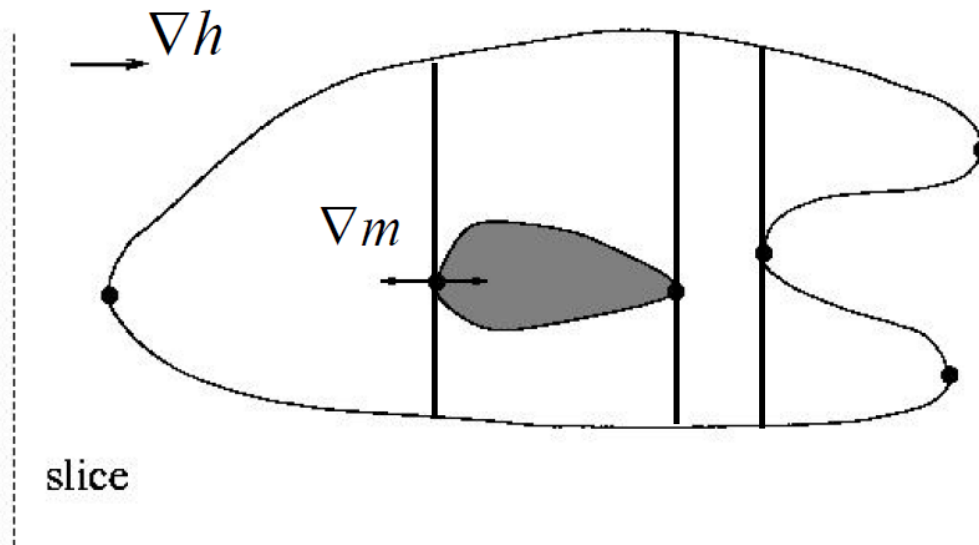
Morse Decomposition

- At a critical point the gradient of the slice function and the obstacle are equal.



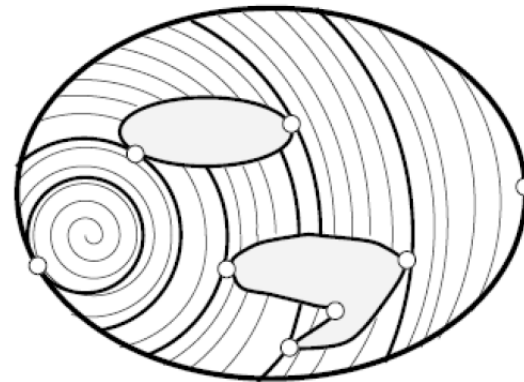
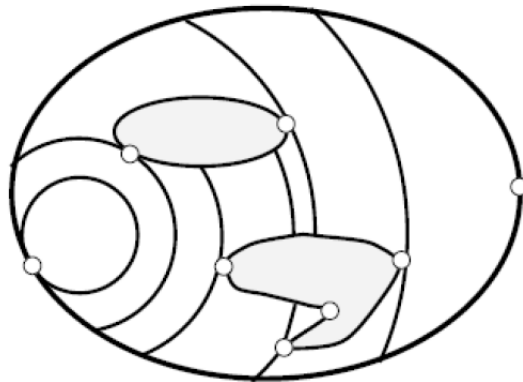
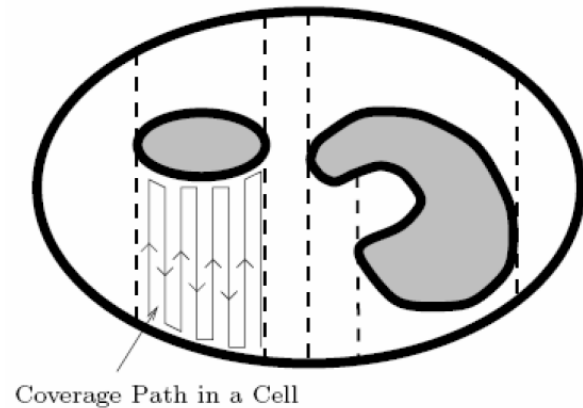
Morse Decomposition

- Each cell can be covered by back and forth motions combined with a contour following



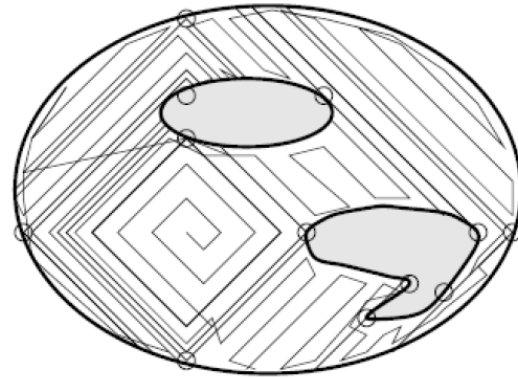
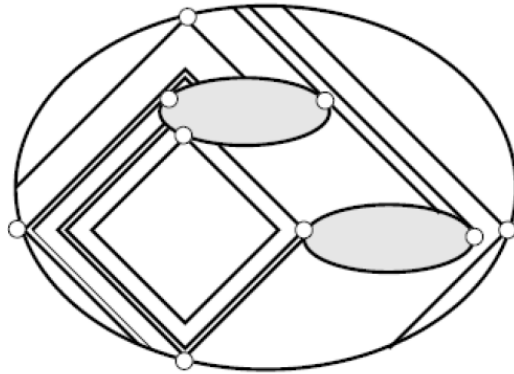
Morse Decomposition

- "Classical" $h(x,y)=x$
- Circular $h(x,y)=x^2+y^2$



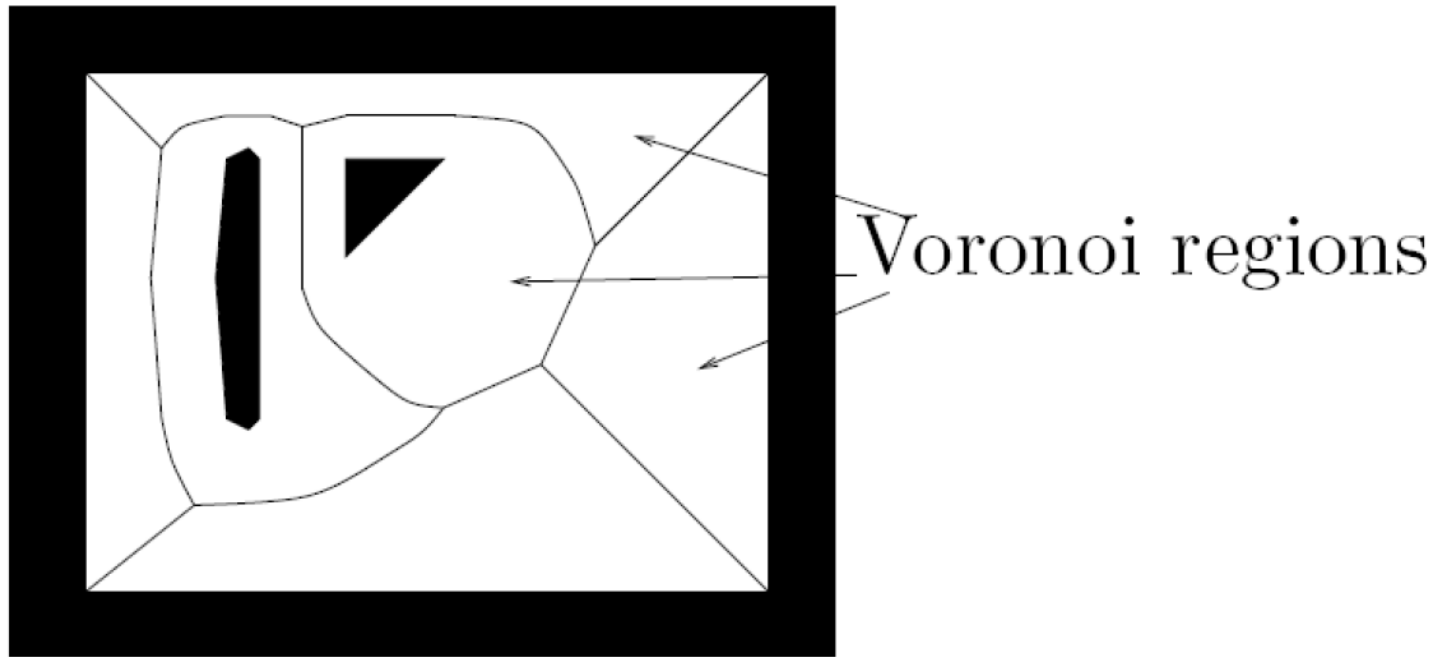
Morse Decomposition

- Squares $h(x,y)=|x|+|y|$



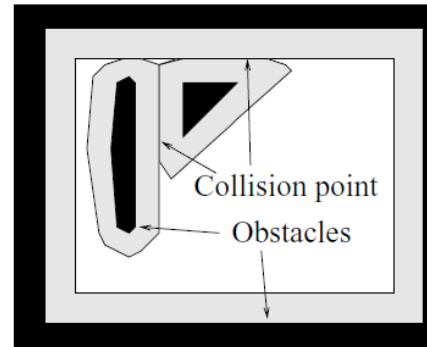
Brushfire Decomposition

- The brushfire algorithm can be used to approximate the Generalized Voronoi Diagram

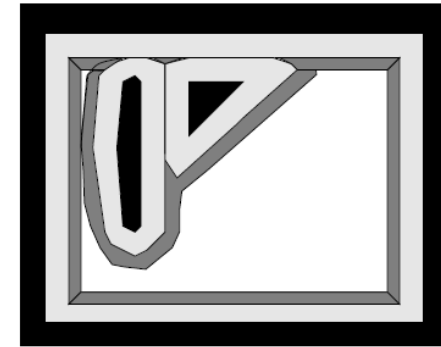


Brushfire Decomposition

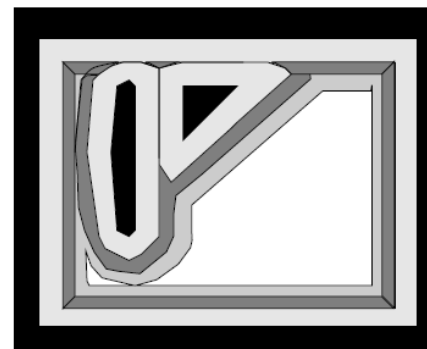
- Follow the boundaries of obstacles.
- Increase distance when reaching a visited point.
- Follow GVD boundary



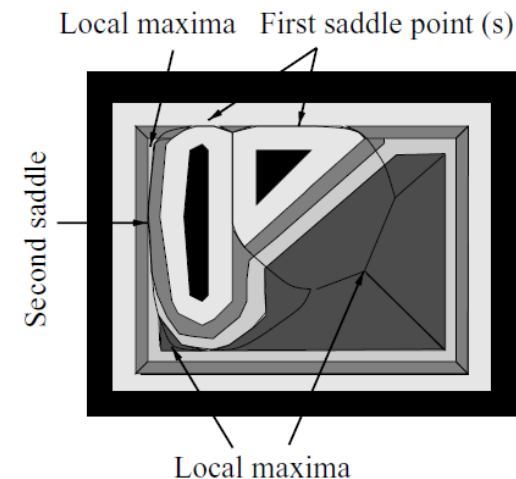
Stage 1



Stage 2



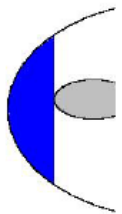
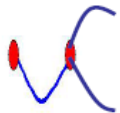
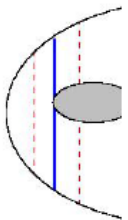
Stage 3





Sensor Based / Incremental Convergence

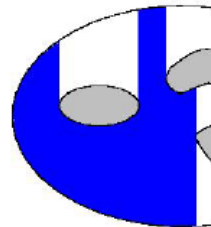
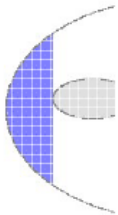
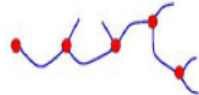
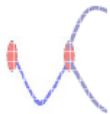
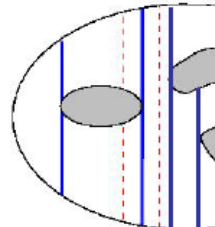
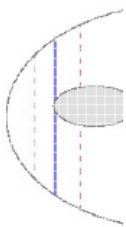
Start covering while looking for critical points



Stage 1



Construct graph (Reeb graph) while covering



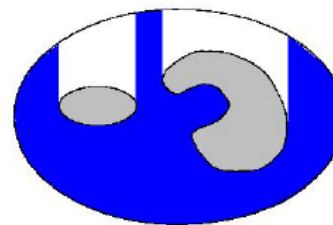
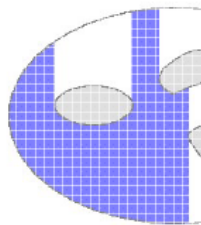
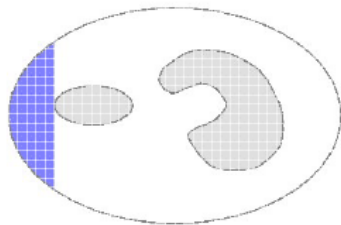
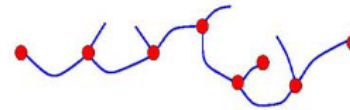
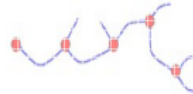
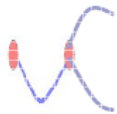
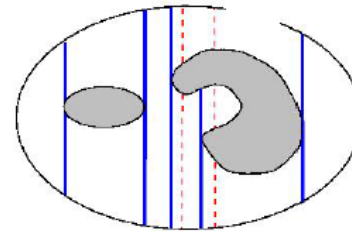
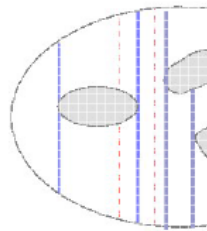
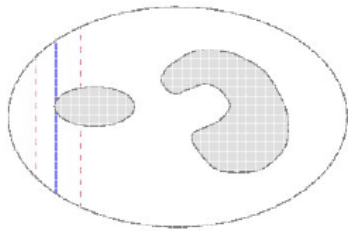
Stage 1

Stage 2



Sensor Based / Incremental Convergence

Continue moving to the right



Stage 1

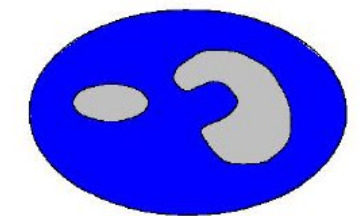
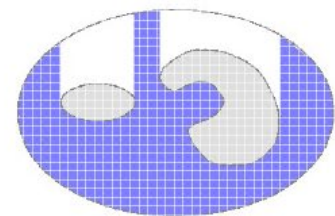
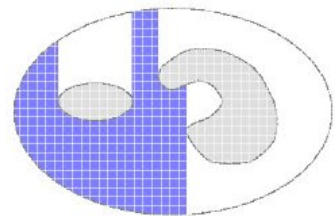
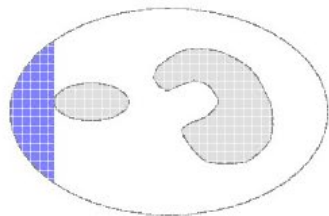
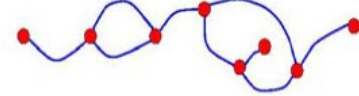
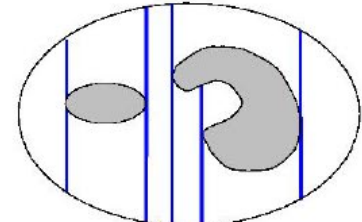
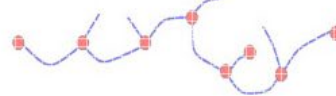
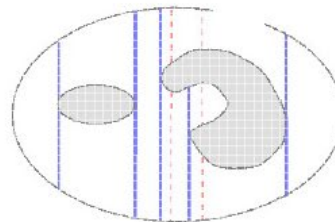
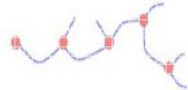
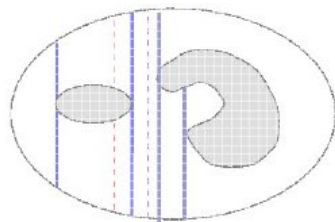
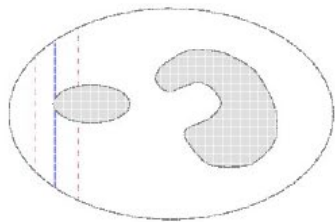
Stage 2

Stage 3



Sensor Based / Incremental Convergence

Reverse and tie up unfinished edges



Stage 1

Stage 2

Stage 3

Stage 4



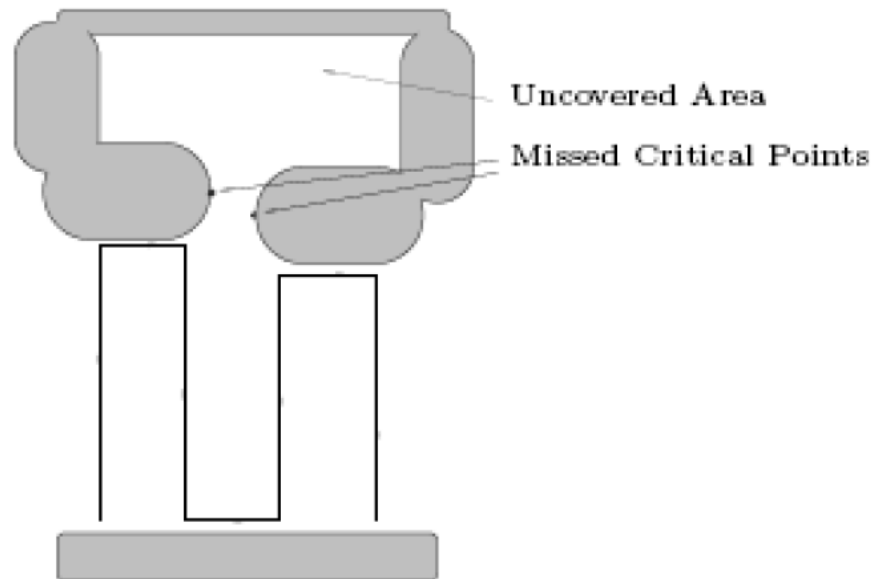
Sensor Based

- Will the sensor based approach always be able to provide full coverage?



Challenge with The Sensor Based Approach

- Due to the width of the coverage with it may miss regions.
- Requires an extra turn to detect closing critical points.



Next lecture(s)

- Lecture 6: Probabilistic planning
- Midterm evaluation

